

Price Optimization

Step 1: Data Collection

Load the data into R:

Load necessary Libraries

```
library(readr)
```

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages —————
```

```
tidyverse 2.0.0 —
```

```
## ✓ dplyr      1.1.4      ✓ purrr      1.0.2
```

```
## ✓ forcats   1.0.0      ✓ stringr   1.5.1
```

```
## ✓ ggplot2    3.5.1      ✓ tibble     3.2.1
```

```
## ✓ lubridate 1.9.3      ✓ tidyr      1.3.1
```

```
## — Conflicts —————
```

```
tidyverse_conflicts() —
```

```
## ✖ dplyr::filter() masks stats::filter()
```

```
## ✖ dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all  
conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(ggplot2)
```

```
library(lmtest)
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
##
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## as.Date, as.Date.numeric
```

```
library(lubridate)
```

```
library(dplyr)
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8

library(ROI)

## ROI: R Optimization Infrastructure
## Registered solver plugins: nlminb.
## Default solver: auto.

library(openxlsx)

# Load the dataset
data <- read_csv("Su24Data (3).csv")

## Rows: 27968 Columns: 6
## — Column specification

```

```
## Delimiter: ","
## chr  (4): Date, Category, Sub-Category, Price
## dbl  (1): Sales Volume
## time (1): Time
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this
message.

# View the first few rows of the data
head(data)

## # A tibble: 6 × 6
##   Date      Time  Category `Sub-Category` Price `Sales Volume`
##   <chr>    <time> <chr>    <chr>         <chr>         <dbl>
## 1 1/1/2021 18:07 Bread    Sourdough    $5.56          2
## 2 1/1/2021 08:47 Bread    Baguette     $4.46          2
## 3 1/1/2021 12:15 Bread    Rye          $6.02          2
## 4 1/1/2021 11:08 Bread    Whole Wheat  $6.18          1
## 5 1/1/2021 09:43 Bread    White        $5.05          1
## 6 1/1/2021 16:05 Cakes    Carrot Cake  $4.51          3
```

Step 2: Data Cleaning and Preparation

Clean the data and handle missing values:

```
# Change the time to 12-hour format
data$Time <- format(strptime(data$Time, format='%H:%M:%S'), '%r')
```

```

# Check for missing values
sum(is.na(data))

## [1] 0

# Remove the '$' symbol
data$Price <- stringr::str_replace(data$Price, '\\$', '')
data$Price <- as.numeric(data$Price)

# Remove outliers (example using IQR)
Q1 <- quantile(data$Price, 0.25)
Q3 <- quantile(data$Price, 0.75)
IQR <- Q3 - Q1
df <- data %>%
  filter(Price >= (Q1 - 1.5 * IQR) & Price <= (Q3 + 1.5 * IQR))

```

Step 3: Exploratory Data Analysis (EDA)

Perform EDA to understand data distribution and relationships:

```

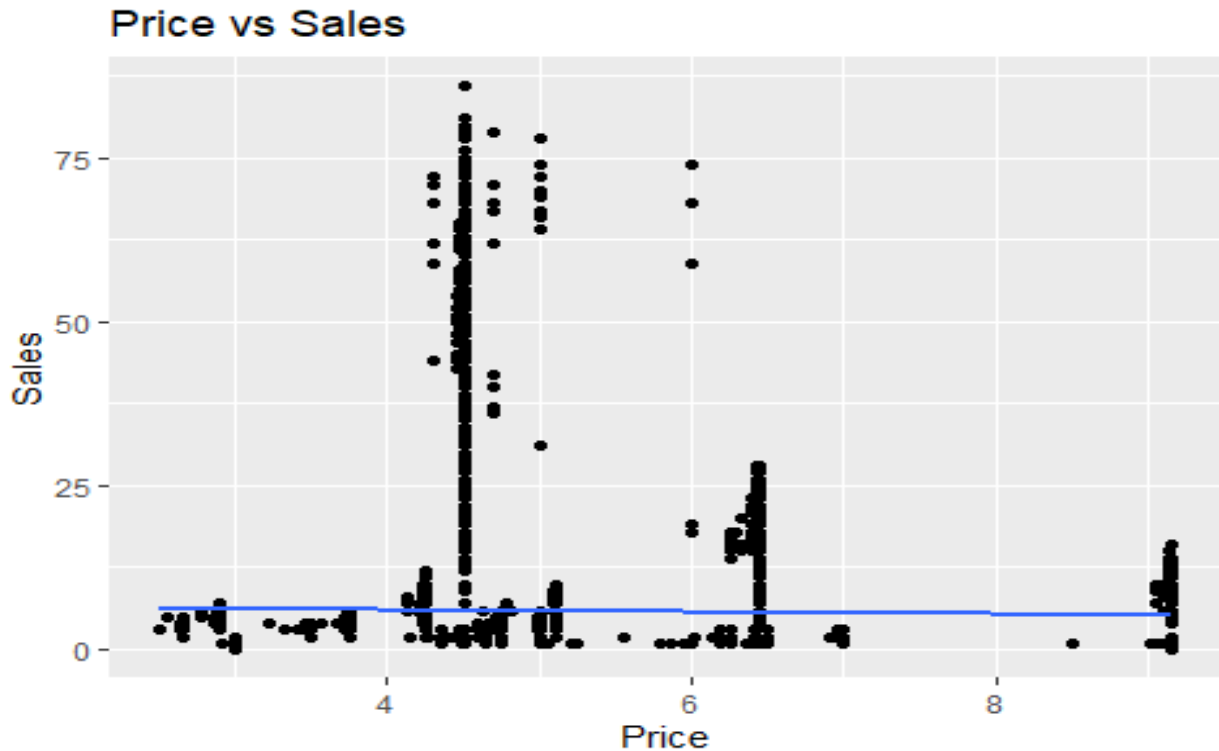
# Summary statistics
summary(data)

##      Date              Time              Category              Sub-Category
## Length:27968          Length:27968          Length:27968          Length:27968
## Class :character      Class :character      Class :character      Class :character
## Mode  :character      Mode  :character      Mode  :character      Mode  :character
##
##
##
##      Price              Sales Volume
## Min.   :2.510          Min.    : 0.000
## 1st Qu.:4.250          1st Qu.: 2.000
## Median :4.650          Median : 3.000
## Mean   :5.143          Mean    : 5.835
## 3rd Qu.:6.440          3rd Qu.: 6.000
## Max.   :9.150          Max.    :86.000

# Visualizations
ggplot(data, aes(x = Price, y = `Sales Volume`)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(title = "Price vs Sales", x = "Price", y = "Sales")

## `geom_smooth()` using formula = 'y ~ x'

```



Correlation matrix

```
cor(data[, 5:6])
```

```
##              Price Sales Volume
## Price          1.00000000 -0.02588448
## Sales Volume -0.02588448  1.00000000
```

Step 4: Model Selection

Choose appropriate models for pricing optimization:

Simple linear regression model

```
model <- lm(`Sales Volume` ~ Price, data = data)
summary(model)
```

```
##
## Call:
## lm(formula = `Sales Volume` ~ Price, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.160 -3.955 -3.046 -0.175  80.067
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.61572    0.19005   34.81  < 2e-16 ***
## Price        -0.15180    0.03506   -4.33  1.5e-05 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.04 on 27966 degrees of freedom
## Multiple R-squared:  0.00067,    Adjusted R-squared:  0.0006343
## F-statistic: 18.75 on 1 and 27966 DF,  p-value: 1.495e-05
```

Step 5: Model Training

Split the data into training and testing sets and train the model:

```
# Split the data
set.seed(123)
train_indices <- sample(seq_len(nrow(data)), size = 0.7*nrow(data))
train_data <- data[train_indices, ]
test_data <- data[-train_indices, ]

# Train the model
model <- lm(`Sales Volume` ~ Price, data = train_data)
summary(model)

##
## Call:
## lm(formula = `Sales Volume` ~ Price, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.128 -3.921 -3.013 -0.143  80.102
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.58817    0.22437   29.36  <2e-16 ***
## Price        -0.15343    0.04125   -3.72   2e-04 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.947 on 19575 degrees of freedom
## Multiple R-squared:  0.0007063,    Adjusted R-squared:  0.0006553
## F-statistic: 13.84 on 1 and 19575 DF,  p-value: 2e-04
```

Step 6: Model Evaluation

Evaluate the model's performance:

```
# Predictions on test data
predictions <- predict(model, newdata = test_data)

# Evaluation metrics
actuals <- test_data$`Sales Volume`
mse <- mean((predictions - actuals)^2)
rmse <- sqrt(mse)
```

```

mae <- mean(abs(predictions - actuals))

cat("MSE:", mse, "\n")
## MSE: 105.16

cat("RMSE:", rmse, "\n")
## RMSE: 10.25476

cat("MAE:", mae, "\n")
## MAE: 5.128035

```

Step 7: Optimization

Optimize pricing using the model:

```

# Define the objective function
objective_function <- function(Price) {
  predict(model, newdata = data.frame(Price = Price))
}

# Use an optimization algorithm to find the optimal price
opt_price <- optimize(objective_function, interval = range(data$Price),
maximum = TRUE)
opt_price

## $maximum
## [1] 2.510063
##
## $objective
##      1
## 6.20306

# Function to perform pricing optimization for a given sub-category
optimize_pricing <- function(sub_data) {
  # Split the data into training and testing sets
  set.seed(123)
  train_indices <- sample(seq_len(nrow(sub_data)), size = 0.7*nrow(sub_data))
  train_data <- sub_data[train_indices, ]
  test_data <- sub_data[-train_indices, ]

  # Train a simple linear regression model
  model <- lm(`Sales Volume` ~ Price, data = train_data)

  # Define the objective function
  objective_function <- function(Price) {
    predict(model, newdata = data.frame(Price = Price))
  }

  # Use an optimization algorithm to find the optimal price

```

```

    opt_price <- optimize(function(Price) -objective_function(Price), interval
= range(sub_data$Price, na.rm = TRUE), maximum = FALSE)

    return(opt_price$minimum)
}

# Perform optimization for each sub-category
results <- data %>%
  group_by(`Sub-Category`) %>%
  summarise(optimal_price = optimize_pricing(cur_data()))

## Warning: There was 1 warning in `summarise()`.
## i In argument: `optimal_price = optimize_pricing(cur_data())`.
## i In group 1: `Sub-Category = "Anisette Biscotti"`.
## Caused by warning:
## ! `cur_data()` was deprecated in dplyr 1.1.0.
## i Please use `pick()` instead.

# View results
print(results)

## # A tibble: 23 × 2
##   `Sub-Category`      optimal_price
##   <chr>              <dbl>
## 1 Anisette Biscotti      3.22
## 2 Baguette              4.46
## 3 Banana Nut Pound      8.51
## 4 Black & White         4.20
## 5 Brownies              4.24
## 6 Cannoli              6.25
## 7 Carrot Cake          4.51
## 8 Chocolate Chip Scone  4.35
## 9 Chocolate Cup Cake    2.93
## 10 Danish              4.13
## # i 13 more rows

# Write the results to an Excel file
write.xlsx(results, "C:/Users/Zachary Patrea/Documents/Price
Opt/PricingSteps1.xlsx", sheetName = "Optimal Price")

```

Step 8: Implementation

Implement the optimal prices and monitor sales performance:

```

# Set the optimal price
optimal_price <- opt_price$maximum

# Monitor sales performance (example code, implementation may vary)
sales_performance <- data %>%
  mutate(predicted_sales = predict(model, newdata = data.frame(Price =
optimal_price)))

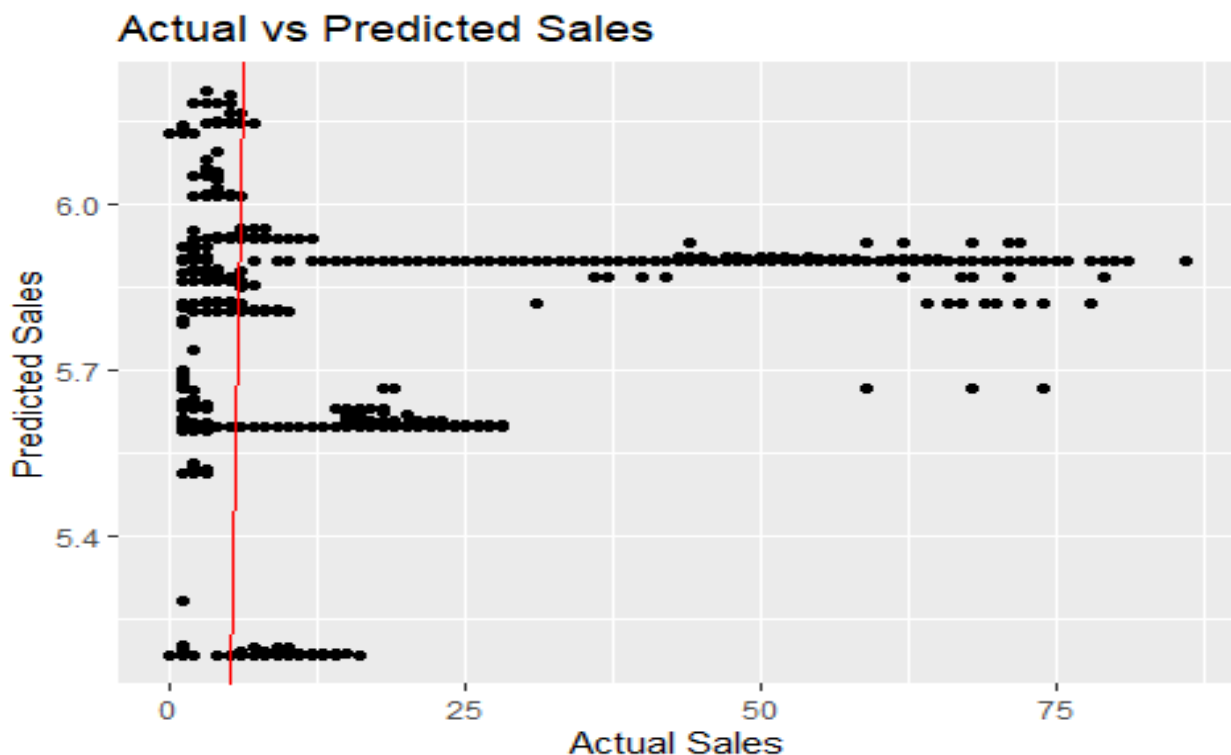
```

Step 9: Evaluation and Feedback

Evaluate the impact and gather feedback for continuous improvement:

```
# Compare actual vs predicted sales
actual_vs_predicted <- data %>%
  mutate(predicted_sales = predict(model, newdata = data))

# Plot actual vs predicted sales
ggplot(actual_vs_predicted, aes(x = `Sales Volume`, y = predicted_sales)) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  labs(title = "Actual vs Predicted Sales", x = "Actual Sales", y =
"Predicted Sales")
```



Excel File

I'll include the steps in the provided CSV file for reference:

```
# Load necessary library
library(openxlsx)

# Add steps to the data
steps <- data.frame(
  Step = c("Data Collection", "Data Cleaning and Preparation", "Exploratory
Data Analysis (EDA)", "Model Selection", "Model Training", "Model
Evaluation", "Optimization", "Implementation", "Evaluation and Feedback"),
```



```

Description = c(
  "Gather historical sales data, including prices, units sold, costs, and
  any other relevant variables.",
  "Clean the data by handling missing values and removing outliers.",
  "Perform EDA to understand data distribution and relationships using
  visualizations and summary statistics.",
  "Select appropriate models for pricing optimization, such as linear
  regression or machine learning models.",
  "Split the data into training and testing sets and train the selected
  model on the training data.",
  "Evaluate the model's performance using metrics such as RMSE, MAE, and
  MSE.",
  "Optimize pricing using the trained model and optimization algorithms.",
  "Implement the optimal prices and monitor sales performance.",
  "Evaluate the impact of the new pricing strategy and gather feedback for
  continuous improvement."
)
)

# Write the steps to an Excel file
write.xlsx(steps, "C:/Users/username/Documents/Price
Opt/Pricing_Optimization_Steps1.xlsx", sheetName = "Step", append = TRUE)

```

By following these steps and using the provided R code, you can conduct pricing optimization for your sales dataset effectively!

...